tence is hallucinated.[3] The following subsections will describe each of the SelfCheckGPT variants.

5.1 SelfCheckGPT with BERTScore

Let $\mathcal{B}(.,.)$ denote the BERTScore between two sentences. SelfCheckGPT with BERTScore finds the average BERTScore of the i-th sentence with the most similar sentence from each drawn sample:

$$\mathcal{S}_{\text{BERT}}(i) = 1 - \frac{1}{N}\sum_{n=1}^{N} \max_k \left(\mathcal{B}(r_i, s_k^n)\right) \quad (1)$$

where r_i represents the i-th sentence in R and s_k^n represents the k-th sentence in the n-th sample S^n. This way if the information in a sentence appears in many drawn samples, one may assume that the information is factual, whereas if the statement appears in no other sample, it is likely a hallucination. In this work, RoBERTa-Large (Liu et al., 2019) is used as the backbone of BERTScore.

5.2 SelfCheckGPT with Question Answering

We also consider using the automatic multiple-choice question answering generation (MQAG) framework (Manakul et al., 2023) to measure consistency for SelfCheckGPT. MQAG assesses consistency by generating multiple-choice questions over the main generated response, which an independent answering system can attempt to answer while conditioned on the other sampled responses. If questions on consistent information are queried, the answering system is expected to predict similar answers. MQAG consists of two stages: question generation $\mathtt{G}$ and question answering $\mathtt{A}$. For the sentence r_i in the response R, we draw questions q and options $\mathbf{o}$:

$$q, \mathbf{o} \sim P_{\mathtt{G}}(q, \mathbf{o}|r_i, R) \quad (2)$$

The answering stage $\mathtt{A}$ selects the answers:

$$a_R = \underset{k}{\text{argmax}} \left[P_{\mathtt{A}}(o_k|q, R, \mathbf{o})\right] \quad (3)$$

$$a_{S^n} = \underset{k}{\text{argmax}} \left[P_{\mathtt{A}}(o_k|q, S^n, \mathbf{o})\right] \quad (4)$$

We compare whether a_R is equal to a_{S^n} for each sample in $\{S^1, ..., S^N\}$, yielding #matches $N_{\mathtt{m}}$ and #not-matches $N_{\mathtt{n}}$. A simple inconsistency score for the i-th sentence and question q based on the match/not-match counts is defined: $\mathcal{S}_{\text{QA}}(i, q) = \frac{N_{\mathtt{n}}}{N_{\mathtt{m}}+N_{\mathtt{n}}}$. To take into account the answerability of generated questions, we show in Appendix B that we can modify the inconsistency score by applying soft-counting, resulting in:

$$\mathcal{S}_{\text{QA}}(i, q) = \frac{\gamma_2^{N'_{\mathtt{n}}}}{\gamma_1^{N'_{\mathtt{m}}} + \gamma_2^{N'_{\mathtt{n}}}} \quad (5)$$

where $N'_{\mathtt{m}}$ = the effective match count, $N'_{\mathtt{n}}$ = the effective mismatch count, with γ_1 and γ_2 defined in Appendix B.1. Ultimately, SelfCheckGPT with QA is the average of inconsistency scores across q,

$$\mathcal{S}_{\text{QA}}(i) = \mathbb{E}_q \left[\mathcal{S}_{\text{QA}}(i, q)\right] \quad (6)$$

5.3 SelfCheckGPT with n-gram

Given samples $\{S^1, ..., S^N\}$ generated by an LLM, one can use the samples to create a new language model that approximates the LLM. In the limit as N gets sufficiently large, the new language model will converge to the LLM that generated the responses. We can therefore approximate the LLM's token probabilities using the new language model.

In practice, due to time and/or cost constraints, there can only be a limited number of samples N. Consequently, we train a simple n-gram model using the samples $\{S^1, ..., S^N\}$ as well as the main response R (which is assessed), where we note that including R can be considered as a smoothing method where the count of each token in R is increased by 1. We then compute the average of the log-probabilities of the sentence in response R,

$$\mathcal{S}_{n\text{-gram}}^{\text{Avg}}(i) = -\frac{1}{J}\sum_j \log \tilde{p}_{ij} \quad (7)$$

where $\tilde{p}_{ij}$ is the probability (of the j-th token of the i-th sentence) computed using the n-gram model. Similar to the grey-box approach, we can also use the maximum of the negative log probabilities,

$$\mathcal{S}_{n\text{-gram}}^{\text{Max}}(i) = \max_j \left(-\log \tilde{p}_{ij}\right) \quad (8)$$

5.4 SelfCheckGPT with NLI

Natural Language Inference (NLI) determines whether a hypothesis follows a premise, classified into either entailment/neutral/contradiction. NLI measures have been used to measure faithfulness in summarization, where Maynez et al. (2020) use a textual entailment classifier trained on MNLI (Williams et al., 2018) to determine if a summary contradicts a context or not. Inspired by NLI-based

[3] With the exception of SelfCheckGPT with n-gram as the score of the n-gram language model is not bounded.

summary assessment, we consider using the NLI contradiction score as a SelfCheckGPT score.

For SelfCheck-NLI, we use DeBERTa-v3-large (He et al., 2023) fine-tuned to MNLI as the NLI model. The input for NLI classifiers is typically the `premise` concatenated to the `hypothesis`, which for our methodology is the sampled passage S^n concatenated to the sentence to be assessed r_i. Only the logits associated with the 'entailment' and 'contradiction' classes are considered,

$$P(\text{contradict}|r_i, S^n) = \frac{\exp(z_c)}{\exp(z_e) + \exp(z_c)} \quad (9)$$

where z_e and z_c are the logits of the 'entailment' and 'contradiction' classes, respectively. This normalization ignores the neutral class and ensures that the probability is bounded between 0.0 and 1.0. The SelfCheckGPT with NLI score for each sample S^n is then defined as,

$$\mathcal{S}_{\text{NLI}}(i) = \frac{1}{N}\sum_{n=1}^{N} P(\text{contradict}|r_i, S^n) \quad (10)$$

5.5 SelfCheckGPT with Prompt

LLMs have recently been shown to be effective in assessing information consistency between a document and its summary in zero-shot settings (Luo et al., 2023). Thus, we query an LLM to assess whether the i-th sentence is supported by sample S^n (as the context) using the following prompt.

```
---------------------------------------------
Context: {}
Sentence: {}
Is the sentence supported by the context above?
Answer Yes or No:
---------------------------------------------
```

Initial investigation showed that GPT-3 (text-davinci-003) will output either `Yes` or `No` 98% of the time, while any remaining outputs can be set to `N/A`. The output from prompting when comparing the i-th sentence against sample S^n is converted to score x_i^n through the mapping {`Yes`: 0.0, `No`: 1.0, `N/A`: 0.5}. The final inconsistency score is then calculated as:

$$\mathcal{S}_{\text{Prompt}}(i) = \frac{1}{N}\sum_{n=1}^{N} x_i^n \quad (11)$$

SelfCheckGPT-Prompt is illustrated in Figure 1. Note that our initial investigations found that less capable models such as GPT-3 (text-curie-001) or LLaMA failed to effectively perform consistency assessment via such prompting.

6 Data and Annotation

As, currently, there are no standard hallucination detection datasets available, we evaluate our hallucination detection approaches by 1) generating synthetic Wikipedia articles using GPT-3 on the individuals/concepts from the WikiBio dataset (Lebret et al., 2016); 2) manually annotating the factuality of the passage at a sentence level; 3) evaluating the system's ability to detect hallucinations.

WikiBio is a dataset where each input contains the first paragraph (along with tabular information) of Wikipedia articles of a specific concept. We rank the WikiBio test set in terms of paragraph length and randomly sample 238 articles from the top 20% of longest articles (to ensure no very obscure concept is selected). GPT-3 (text-davinci-003) is then used to generate Wikipedia articles on a concept, using the prompt "`This is a Wikipedia passage about {concept}:`". Table 1 provides the statistics of GPT-3 generated passages.

#Passages	#Sentences	#Tokens/passage
238	1908	184.7$\pm$36.9

Table 1: The statistics of **WikiBio GPT-3 dataset** where the number of tokens is based on the OpenAI GPT-2 tokenizer.

We then annotate the sentences of the generated passages using the guidelines shown in Figure 3 such that each sentence is classified as either:

- **Major Inaccurate** (Non-Factual, **1**): The sentence is entirely hallucinated, i.e. the sentence is unrelated to the topic.
- **Minor Inaccurate** (Non-Factual, **0.5**): The sentence consists of some non-factual information, but the sentence is related to the topic.
- **Accurate** (Factual, **0**): The information presented in the sentence is accurate.

Of the 1908 annotated sentences, 761 (39.9%) of the sentences were labelled major-inaccurate, 631 (33.1%) minor-inaccurate, and 516 (27.0%) accurate. 201 sentences in the dataset had annotations from two different annotators. To obtain a single label for this subset, if both annotators agree, then the agreed label is used. However, if there is disagreement, then the worse-case label is selected (e.g., {minor inaccurate, major inaccurate} is mapped to major inaccurate). The inter-annotator agreement, as measured by Cohen's κ (Cohen, 1960), has κ

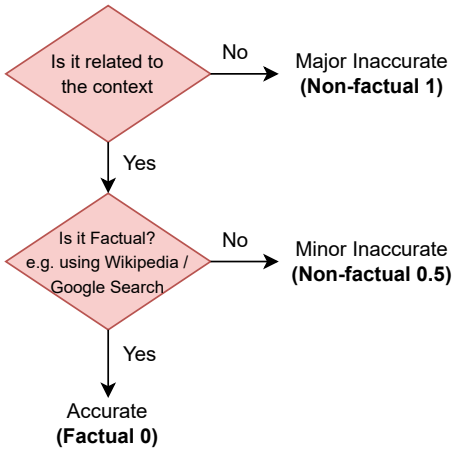

Figure 3: Flowchart of our annotation process

values of 0.595 and 0.748, indicating *moderate* and *substantial* agreement (Viera et al., 2005) for the 3-class and 2-class scenarios, respectively.[4]

Furthermore, passage-level scores are obtained by averaging the sentence-level labels in each passage. The distribution of passage-level scores is shown in Figure 4, where we observe a large peak at +1.0. We refer to the points at this peak as *total hallucination*, which occurs when the information of the response is unrelated to the real concept and is entirely fabricated by the LLM.

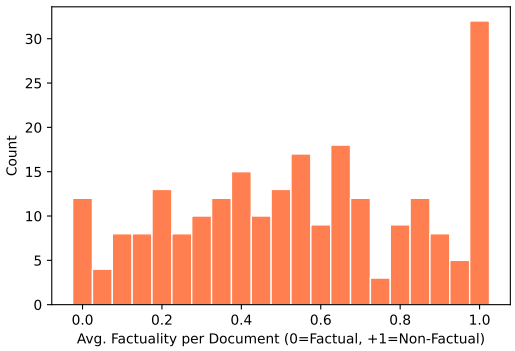

Figure 4: Document factuality scores histogram plot

7 Experiments

The generative LLM used to generate passages for our dataset is **GPT-3** (text-davinci-003), the state-of-the-art system at the time of creating and annotating the dataset. To obtain the main response, we set the temperature to 0.0 and use standard beam search decoding. For the stochastically generated samples, we set the temperature to 1.0 and generate N=20 samples. For the proxy LLM approach, we use LLaMA (Touvron et al., 2023), one of the best-performing open-source LLMs currently available. For SelfCheckGPT-Prompt, we consider both GPT-3 (which is the same LLM that is used to generate passages) as well as the newly released ChatGPT (gpt-3.5-turbo). More details about the systems in SelfCheckGPT and results using other proxy LLMs can be found in the appendix.

7.1 Sentence-level Hallucination Detection

First, we investigate whether our hallucination detection methods can identify the factuality of sentences. In detecting non-factual sentences, both major-inaccurate labels and minor-inaccurate labels are grouped together into the *non-factual* class, while the *factual* class refers to accurate sentences. In addition, we consider a more challenging task of detecting major-inaccurate sentences in passages that are *not* total hallucination passages, which we refer to as *non-factual**.[5] Figure 5 and Table 2 show the performance of our approaches, where the following observations can be made:

1) LLM's probabilities p correlate well with factuality. Our results show that probability measures (from the LLM generating the texts) are strong baselines for assessing factuality. Factual sentences can be identified with an AUC-PR of 53.97, significantly better than the random baseline of 27.04, with the AUC-PR for hallucination detection also increasing from 72.96 to 83.21. This supports the hypothesis that when the LLMs are uncertain about generated information, generated tokens often have higher uncertainty, paving a promising direction for hallucination detection approaches. Also, the probability p measure performs better than the entropy $\mathcal{H}$ measure of top-5 tokens.

2) Proxy LLM perform noticeably worse than LLM (GPT-3). The results of proxy LLM (based on LLaMA) show that the entropy $\mathcal{H}$ measures outperform the probability measures. This suggests that using richer uncertainty information can improve factuality/hallucination detection performance, and that previously the entropy of top-5 tokens is likely to be insufficient. In addition, when using other proxy LLMs such as GPT-NeoX or OPT-30B, the performance is near that of the random baseline. We believe this poor performance occurs as different LLMs have different generating patterns, and so even common tokens may have a

[4] 3-class refers to when selecting between accurate, minor inaccurate, major inaccurate. 2-class refers to when minor/major inaccuracies are combined into one label.

[5] There are 206 non-factual* passages (1632 sentences).